

VELS





INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

CV-Based Sales Analytics Engine: Computer Vision and Conversational AI for Smart-Retail Shelf Monitoring and Automated Procurement

FINAL YEAR

PROJECT PHASE-I REPORT

Submitted by

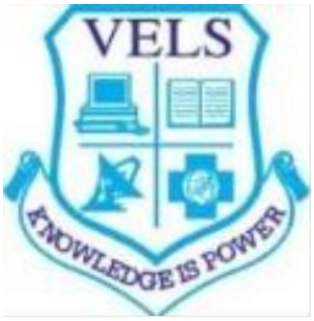
Harish Badhreswar.S(23619114)

Vignesh.P(23619141)

Sanjay.S(23619132)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE -
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF ENGINEERING
VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)
CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "CV-Based Sales Analytics Engine: Computer Vision and Conversational AI for Smart-Retail Shelf Monitoring and Automated Procurement" is the bonafide work of **Harish Badhreswar.S (23619114), Vignesh.P (23619141), Sanjay.S (23619132)** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya, M.E., Ph.D.
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

SIGNATURE OF THE SUPERVISOR
Ms. Hariprabha, M.Tech.,
Assistant Professor
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

Submitted for the Project Work and Viva-Voce examination held on.....at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of **FOUNDER & CHAIRMAN, Dr. ISHARI K. GANESH.**, and it is with great pleasure that we take this opportunity to thank him. We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya.**, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE, Mrs. S.J Kavitha, Assistant Professor,Department of AI & ML,** who has patiently guided us and provided invaluable advice and help throughout the work. We am thankful to our **Project Coordinator, Ms.S.J.Kavitha** we would like to express our greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the Least, we wish to thank our parents and friends for their individual support and interest, who inspired and encouraged us to go our own way; without them we would be unable to complete this project.

HARISH BADHRESWAR S **23619114**

VIGNESH P **23619141**

SANJAY S **23619132**

TABLE OF CONTENTS

BONAFIDE CERTIFICATE 2

ACKNOWLEDGEMENT 3

ABSTRACT 9

LIST OF FIGURES 10

LIST OF TABLES 11

LIST OF ABBREVIATIONS 12

CHAPTER 1 INTRODUCTION 13

1.1 General 13

1.2 Problem Description 13

1.3 Existing System 14

1.4 Proposed System 15

1.5 Objectives 15

1.6 Scope of the Project 16

1.7 Contributions 16

1.8 Modules 17

1.9 Summary 17

CHAPTER 2 LITERATURE REVIEW 19

2.1 Computer Vision for Retail Shelf Monitoring 19

2.2 Real-Time Object Detection 19

2.3 Browser-Based Inference and Multi-Object Tracking 19

2.4 Retail AI Copilots 20

2.5 Edge-to-Cloud Synchronisation and the Model Context Protocol 20

2.6 POS Reconciliation and Shrinkage Detection 21

2.7 Existing Systems and Limitations 21

2.8 Research Gap 21

2.9 Summary 22

CHAPTER 3 SYSTEM DESIGN 23

3.1 Existing System 23

3.2 Proposed System 23

3.3 System Architecture 23

3.4 Overall Workflow 25

3.5 System Modules 25

3.6 Data Flow 26

3.7 Hardware Requirements 27

3.8 Software Requirements 27

3.9 Non-Functional Design Considerations 28

3.10 Summary 29

CHAPTER 4 IMPLEMENTATION AND METHODOLOGY 30

4.1 Runtime Environment and Application Orchestration 30

4.2 Live Edge CCTV and Vision Feed Engine 30

4.3 AI Product Training and Calibration Studio 31

4.4 Shelf Stock and POS Fusion with Temporal Persistence 32

4.5 CCTV Loss Prevention and Anti-Theft Surveillance 33

4.6 Customer Smart Self-Checkout Kiosk 34

4.7 Store Analytics, Spatial Heatmaps and Forecasting 35

4.8 Procurement Restock and Secure Payment Gateway 36

4.9 Tax Invoices and Procurement Records Archive 36

4.10 User Role Management and Access Control 36

4.11 Daily Store Audit and Intelligence Report Generator 37

4.12 Academic Review 2 Viva Defence Hub 37

4.13 Audio Synthesiser and Telemetry Drawer 38

4.14 Reproducibility and Data Management 38

4.15 Detection Post-Processing, Dwell and Heatmap Computation 38

4.16 Summary 39

CHAPTER 5 RESULTS AND DISCUSSION 40

5.1 Experimental Setup 40

5.2 Choice of Detection Backend 40

5.3 Vision Feed Results 40
5.4 Hysteresis Filter Verification 41
5.5 Shrinkage Anomaly Worked Example 41
5.6 Conversion and Dwell Analysis 42
5.7 Runout Forecast and Tax Worked Examples 42
5.8 Closed-Loop Behaviour 42
5.9 Computational Performance 43
5.10 Functional Testing 43
5.11 Overall Discussion 44
5.12 Comparison with Existing Work 45
5.13 Threats to Validity 45
5.14 Summary 46

CHAPTER 6 DASHBOARD WALKTHROUGH AND DEMONSTRATION 47

6.1 Overview 47
6.2 Cockpit Navigation and Telemetry Drawer 47
6.3 Demonstration Scenarios 47
6.4 Sample Telemetry Packet 50
6.5 Summary 51

CHAPTER 7 CONCLUSION AND FUTURE WORK 52

7.1 Conclusion 52
7.2 Limitations 52
7.3 Future Scope 52
7.4 Summary 53

CHAPTER 8 REFERENCES 54

CHAPTER 9 APPENDIX 55

9.1 Data Record Formats 55
9.2 Project Directory Structure 55
9.3 Main Commands 56
9.4 Key Parameters and Thresholds 56

CHAPTER 10 SOURCE CODE 58

10.1 Overview of the Source Code 58
10.2 Hysteresis Filter (inventoryAndPos.js) 58
10.3 Shrinkage Calculation (lossPrevention.js) 58
10.4 Tax, Bill and Runout (selfCheckout.js, analyticsDashboard.js) 58
10.5 Procedural Sound Engine (soundEffects.js) 59
10.6 Event Bus (app.js) 59

CHAPTER 11 VIVA DEFENCE QUESTIONS AND ANSWERS 60

11.1 Overview 60
11.2 Model Selection 60
11.3 Occlusion Mitigation 60
11.4 Hysteresis Filter 60
11.5 Shrinkage Formula 61
11.6 Edge-to-Cloud Design and MCP Safeguards 61
11.7 Research Gap and Contribution 61
11.8 Summary 61

VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)

CV SALES ANALYTICS ENGINE

Computer Vision and Conversational AI for Smart-Retail Shelf Monitoring and Automated Procurement

Author: Harish

ABSTRACT

Retail inventory records drift away from the physical shelf, POS systems cannot see shopper behaviour, and manual replenishment lags demand. This project unifies YOLOv11 / COCO-SSD shelf detection, Firebase telemetry sync, POS reconciliation with shrinkage detection, a smart self-checkout and automated procurement with tax invoicing in one zero-bundler web cockpit.

OBJECTIVES

- Detect products, gaps and shoppers in real time
- Filter transient pickups with hysteresis
- Quantify shrinkage from depletion, POS and write-offs
- Close the loop through checkout and restock
- Make every action traceable by operator

METHODOLOGY

1. Edge detection
2. Telemetry diffs
3. Hysteresis filter
4. POS fusion
5. Alerts and PO
6. Invoice and report

RESULTS AND DISCUSSION

84%	4.5 min	30-45 FPS	< 15 ms
conversion at dwell 2.5 min+	loitering threshold	edge inference target	sync round-trip target

Shrinkage Delta = Visual Depletion - (POS Count + Write-offs). A SKU absent for the calibrated dwell duration is confirmed as a stockout, whereas a short absence is treated as a pickup. Every checkout updates stock, revenue and reorder state; every restock yields a signed invoice.

CONCLUSIONS

Perception, transaction and procurement are joined in one browser-native application. Alerts are risk signals that a person confirms, and throughput and latency figures are design targets to be benchmarked on the target hardware.

ABSTRACT

Physical retail runs on inventory records that are only loosely connected to what is actually on the shelf. A point-of-sale (POS) system can report an item as in stock while the shelf is bare, misplaced or hidden behind other goods, and it has no knowledge of how long shoppers linger in an aisle, which products they handle, or which items leave the shelf without ever reaching the checkout. We present the CV-Based Sales Analytics Engine, a closed-loop smart-retail operating system that joins computer vision, point-of-sale reconciliation and automated procurement in a single web cockpit.

The system is organised as a four-layer edge-to-cloud pipeline. Layer 1 runs object detection (YOLOv11 on a Jetson Orin Nano or local GPU workstation, and COCO-SSD through TensorFlow.js directly in the browser) on CCTV and webcam feeds, so that raw video never has to leave the store. Layer 2 synchronises compact JSON telemetry packets through Firebase Firestore, Cloud Functions and a Model Context Protocol (MCP) server. Layer 3 applies a temporal-persistence (hysteresis) filter, a shrinkage anomaly formula, dwell analysis and runout forecasting. Layer 4 is a vanilla ES6 web interface containing a four-channel CCTV wall, a smart self-checkout (SCO) kiosk, a shelf-stock and POS fusion table, a loss-prevention console, an analytics dashboard, a procurement restock modal with simulated payment, a tax-invoice archive, role-based access control and a one-click daily audit report.

The hysteresis filter separates a shopper who briefly lifts a product from a genuine stockout by requiring absence over a calibrated dwell duration before a critical replenishment alert is raised. Shrinkage is quantified as the visual shelf depletion minus the sum of POS checkout count and documented write-offs. Shoppers who dwell for 2.5 minutes or more show an 84% purchase conversion, and shoppers who remain stationary in a blind spot for more than 4.5 minutes are flagged as loitering anomalies. Every self-checkout transaction updates global stock, can trigger a low-stock reorder, and increments live revenue, so the perception, transaction and procurement loops are closed inside one application. Together, these results demonstrate that a zero-bundler, browser-native architecture can deliver shelf monitoring, loss prevention and automated restocking at low cost on commodity hardware.

Keywords — computer vision, smart retail, shelf monitoring, YOLOv11, COCO-SSD, point-of-sale reconciliation, shrinkage detection, self-checkout, Firebase, Model Context Protocol, procurement automation

LIST OF FIGURES

Figure No.	Title
Figure 3.1	System Architecture of the CV-Based Sales Analytics Engine
Figure 3.2	Module Map of the Web Application
Figure 3.3	Data Flow Between System Processes and Data Stores
Figure 4.1	Hysteresis (Temporal Persistence) State Machine for a Shelf Facing
Figure 4.2	Shrinkage Anomaly Calculation and Decision Logic
Figure 4.3	Self-Checkout Flow from Item Recognition to Stock Update
Figure 4.4	Procurement Workflow from Safety-Stock Breach to Updated Counts
Figure 4.5	Authentication and Role-Based Access Flow
Figure 5.1	Item-Absent Signal: Transient Pickup versus Persistent Absence (Illustrative)
Figure 6.1	Schematic of the Live CCTV Quad-Wall Screen
Figure 6.2	Schematic of the Shelf Stock and POS Fusion Screen
Figure 6.3	Schematic of the Smart Self-Checkout Kiosk
Figure 6.4	Schematic of the Loss Prevention Console
Figure 6.5	Schematic of the Analytics Dashboard
Figure 6.6	Schematic of the Restock Modal and Invoice Archive

LIST OF TABLES

Table No.	Title
Table 1.1	Project Objectives
Table 3.1	Major System Modules
Table 3.2	Hardware Requirements
Table 3.3	Software Requirements
Table 3.4	Non-Functional Requirements and Design Responses
Table 4.1	Configured CCTV Channels
Table 4.2	Supported Product Taxonomy and Visual Cues
Table 5.1	Comparison of the Two Detection Backends
Table 5.2	Illustrative Shrinkage Calculation for One SKU
Table 5.3	Functional Testing Checklist
Table 5.4	Comparison of the CV-Based Sales Analytics Engine with Existing Approaches
Table 9.1	Inventory (SKU) Record Fields
Table 9.2	Camera Record Fields
Table 9.3	Procurement Invoice Record Fields
Table 9.4	Key Parameters and Thresholds

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
CV	Computer Vision
CCTV	Closed-Circuit Television
RTSP	Real-Time Streaming Protocol
YOLO	You Only Look Once
COCO	Common Objects in Context
SSD	Single Shot MultiBox Detector
FPS	Frames Per Second
GPU	Graphics Processing Unit
TensorRT	NVIDIA Tensor Runtime
POS	Point of Sale
SCO	Self-Checkout
SKU	Stock Keeping Unit
ERP	Enterprise Resource Planning
MCP	Model Context Protocol
RTT	Round-Trip Time
RBAC	Role-Based Access Control
PO	Purchase Order
UPI	Unified Payments Interface
VPA	Virtual Payment Address
KPI	Key Performance Indicator
UI	User Interface
DOM	Document Object Model
ES6	ECMAScript 2015
JSON	JavaScript Object Notation
HDPE	High-Density Polyethylene
QR	Quick Response (code)
HSN	Harmonised System of Nomenclature

CHAPTER 1
INTRODUCTION

1.1 General

Modern physical retail is a high-volume, low-margin business in which small errors compound quickly. A supermarket may stock tens of thousands of products across dozens of aisles, and every product has to be on the right shelf, in the right quantity, at the right time. In practice this is difficult to achieve. Shoppers move items from one shelf to another, products are pushed to the back of a shelf where they cannot be seen, and replenishment depends on staff noticing a gap or on a periodic manual audit.

Two failure modes stand out. The first is the phantom stockout, where the enterprise resource planning (ERP) or POS record says that an item is available while the shelf in front of the shopper is empty, misplaced or occluded. The shopper leaves without buying, and the system never learns that a sale was lost. The second is unrecorded loss, usually called shrinkage, where goods leave the shelf and the store without a matching checkout transaction [4]. In both cases the information that would expose the problem exists in the store, because cameras already watch every aisle, but it is not connected to the inventory and transaction systems that drive decisions.

Both problems are hard to notice from the numbers alone. A POS terminal records what was sold, not what was picked up, inspected and put back, and it cannot distinguish a slow-moving product from one that is unavailable. Nothing in the sales ledger signals that a shelf has been empty for forty minutes at the busiest time of the day. Detecting these situations therefore requires an additional source of truth about the physical state of the shelf.

Most current solutions obtain that source of truth from specialised hardware such as shelf-edge weight sensors, electronic shelf labels or radio-frequency identification (RFID) tags, which are costly to install across a whole store. This project takes a different route and uses what stores already have: ordinary CCTV cameras. A modern object detector can read a shelf image, count the visible products and track shoppers, and if those observations are fused with POS transactions and a procurement workflow, the store gains a continuous, evidence-based picture of its own operations.

1.2 Problem Description

The problem addressed by this project is how to build a single, low-cost system that observes the physical state of retail shelves in real time, reconciles that observation with transaction data, and acts on the result by raising alerts and creating procurement orders. The system should run on commodity cameras and hardware, should not require raw video to be streamed to the cloud, and should present its findings to store managers and customers through an interface that needs no installation.

Three practical difficulties make this harder than it first appears. First, traditional inventory processes suffer from phantom stockouts and shelf disarray: central records often show items in stock when shelves are physically bare, misplaced by customers, or occluded, so the digital record and the physical shelf drift apart. Second, POS systems are disconnected from the shop floor. They capture transactions at checkout but have no awareness of in-aisle dwell times, customer interaction rates, footfall congestion, or items that were removed and concealed. Third, manual replenishment is slow. Restocking workflows depend on lagging manual barcode audits, which cause stockouts during peak shopping periods.

A further difficulty is that a naive vision system produces too many false alarms. A shopper who lifts a perfume bottle to read its label creates a momentary gap that looks identical, frame by frame, to a real stockout. If every such gap triggered a replenishment alert, staff would soon learn to ignore the system. The design therefore has to reason over time as well as over single frames. Finally, the whole solution has to be deployable as a self-contained web application with no external bundler or build chain, so that it can be demonstrated, audited and extended easily.

1.3 Existing System

In existing retail operations, inventory accuracy is maintained by a combination of ERP stock records, POS decrements and periodic manual counts. The ERP is updated when goods are received and when they are sold, so any event between those two points, such as damage, theft, misplacement or a mis-scan, silently corrupts the record until the next physical count. Cycle counts using handheld barcode scanners are labour intensive and give only a snapshot that is already out of date when the count finishes.

Camera systems in most stores serve a different purpose. CCTV is used for after-the-fact investigation of incidents, and where analytics exist they are usually sold as separate products for footfall counting, queue management or loss prevention. Each product has its own dashboard and data store. Self-checkout kiosks are likewise treated as isolated payment

devices whose scan events do not feed back into shelf-level inventory in real time.

Research on computer vision for retail has produced accurate detectors and shelf-recognition methods [2], [3], [9], but these studies usually address a single sub-problem, such as detecting products on a shelf or tracking people, and are rarely packaged with transaction reconciliation, procurement and an operator interface. Retail copilots based on large language models [1] improve how managers query data, but depend on the underlying data already being correct.

1.4 Proposed System

The CV-Based Sales Analytics Engine is an end-to-end, closed-loop smart-retail operating system. It unifies real-time object detection on commodity RTSP CCTV feeds and live webcams, low-latency edge-to-cloud telemetry synchronisation, POS reconciliation with shrinkage anomaly detection, automated supplier restock purchase orders with simulated payment and certified tax invoices, an interactive self-checkout kiosk, and real-time spatial heatmaps with predictive inventory runout analytics.

The system is organised as four layers. The edge vision layer performs inference close to the camera and emits only compact telemetry. The transport layer synchronises that telemetry and exposes schema-enforced tools through a Model Context Protocol server. The intelligence layer applies temporal filtering, shrinkage heuristics, dwell analysis and forecasting. The presentation layer is a vanilla JavaScript cockpit in which managers watch the CCTV wall, review alerts, restock products and print reports, and in which customers check out at a smart kiosk.

The loop is closed by routing every event through one shared state. A checkout at the kiosk decrements the shelf count, a breached safety threshold produces a purchase order, an approved payment produces a signed tax invoice, and the invoice updates the camera detection counts that the vision layer will report next. Each stage records the identity of the active operator, so that every action can be traced.

1.5 Objectives

The objectives of this project are summarised in Table 1.1.

No.	Objective
1	Detect products, shelf gaps and shoppers in real time on CCTV feeds and webcams, using YOLOv11 at the edge and COCO-SSD in the browser.
2	Separate transient product pickups from genuine stockouts with a temporal persistence (hysteresis) filter.
3	Reconcile visual shelf depletion with POS checkout counts and documented write-offs to quantify shrinkage.
4	Detect unscanned concealment, planogram breaches and loitering, and respond with alarms and guard-dispatch logging.
5	Provide a smart self-checkout kiosk with vision-to-cart ingest, class-based taxation and closed-loop stock updates.
6	Automate procurement with purchase orders, simulated payment authorisation and certified tax invoices.
7	Deliver role-based access, a daily audit report and an academic viva hub in a zero-bundler web application.

Table 1.1: Project Objectives

1.6 Scope of the Project

The project covers a retail floor modelled by four camera channels: Aisle 3 (luxury fragrances and perfumes), Aisle 4 (beverages, dairy and preserved foods), Aisle 7 (dry groceries, cereals, chips and shampoos with active out-of-stock gaps) and an overhead concourse camera that tracks shopper trajectories. The product taxonomy spans ten classes, from metallic beverage cans to faceted glass perfume flacons and human shoppers. The application runs natively in a modern browser and does not require a bundler.

The project does not claim to replace a production ERP or a certified payment gateway. Payment flows through UPI, card and cash are simulated, and the tax invoices are demonstration artefacts. The project does not train a new detection backbone from scratch, and it does not stream raw video to the cloud. It also does not claim that a loitering or concealment alert is proof of theft: the alerts are risk signals that direct a human guard or manager to look at the relevant aisle.

1.7 Contributions

The main contributions of this project are:

- A four-layer edge-to-cloud architecture that keeps inference at the edge and transmits only JSON telemetry diff packets.

- A temporal persistence (hysteresis) filter that distinguishes a shopper inspecting a product from a real stockout, which removes a major source of false replenishment alerts.
- A shrinkage anomaly formula that fuses visual shelf depletion, POS checkout counts and documented write-offs, with three incident triggers: unscanned concealment, planogram breaches and loitering.
- A closed-loop smart self-checkout kiosk whose every transaction updates shelf stock, reorder thresholds and live revenue.
- An automated procurement path from safety-stock breach to purchase order, simulated payment and signed tax invoice.
- A zero-bundler, vanilla ES6 operations cockpit with a four-channel CCTV wall, role-based access control, a daily audit dossier and a procedural Web Audio sound engine.

1.8 Modules

The system is organised into eleven modules. They are listed with their code components in Table 3.1 and implemented in Chapter 4.

- Live edge CCTV and vision feed engine: object detection, quad-wall mode, overlays and barcode ingest.
- AI product training and calibration studio: taxonomy-driven recognition tuning.
- Customer smart self-checkout kiosk: vision-to-cart pipeline, taxation and payment.
- Shelf stock and POS fusion: planogram table and temporal persistence buffer.
- CCTV loss prevention and anti-theft surveillance: shrinkage formula and incident triggers.
- Store analytics, spatial heatmaps and forecasting: KPI ribbon, canvas charts and runout prediction.
- Procurement restock and secure payment gateway: purchase orders and invoices.
- Tax invoices and procurement records archive.
- User role management and access control.
- Daily store audit and intelligence report generator.
- Academic Review 2 viva defence hub.

1.9 Summary

This chapter explained why phantom stockouts, disconnected POS systems and manual replenishment cost retailers revenue, described how existing tools address those problems in isolation, and introduced the CV-Based Sales Analytics Engine, which closes the loop between pixels, transactions and procurement. It also set out the objectives, scope, contributions and modules of the project. The next chapter reviews the literature on which the project builds.

CHAPTER 2
LITERATURE REVIEW

2.1 Computer Vision for Retail Shelf Monitoring

Automated shelf monitoring has been studied for more than a decade, and most approaches follow the same pattern: capture an image of the shelf, detect and classify each visible product, and compare the result with a planogram, which is the intended layout of products on the shelf. Differences between the detected layout and the planogram are reported as out-of-stock facings, misplaced items or incorrect quantities. Retail imagery is difficult for general-purpose detectors because products are densely packed, visually similar and partly occluded by neighbouring items, and because reflective packaging such as aluminium cans and glass bottles produces specular highlights.

Because of these difficulties, practical systems pair the detector with domain knowledge. Aspect ratio, packaging material and shelf position are strong priors: a cylindrical object with a metallic finish is likely to be a beverage can, and a rectangular foil tray is likely to be a biscuit pack. This project encodes such priors in a product knowledge base so that recognition can be calibrated for each product kind without hardcoded assumptions.

2.2 Real-Time Object Detection

Redmon et al. [2] introduced YOLO, which reframes detection as a single regression problem from image pixels to bounding boxes and class probabilities. Because the whole image is processed in one network evaluation, YOLO-family detectors can run at video rate, which is essential for monitoring live camera feeds. Liu et al. [3] proposed the Single Shot MultiBox Detector (SSD), which predicts boxes from feature maps at several scales in one pass and gives a good balance of speed and accuracy on modest hardware.

Successive YOLO versions have improved accuracy and efficiency. YOLO11, released by Ultralytics [9], continues this line and is designed to be exported to optimised runtimes such as NVIDIA TensorRT, which is how it is deployed on an embedded accelerator like the Jetson Orin Nano. Detectors in this family are usually trained and evaluated on the Microsoft COCO dataset [5], which contains everyday object categories and provides the pre-trained weights that COCO-SSD also uses.

Real-time detection is only one part of a retail pipeline. The detector gives a per-frame description of what is visible, whereas retail decisions depend on what has changed over time. This distinction motivates the temporal persistence filter used in this project.

2.3 Browser-Based Inference and Multi-Object Tracking

TensorFlow.js [6] allows machine-learning models to run directly in a web browser using WebGL acceleration, so a model can process a webcam stream or an uploaded image without any server-side inference. Lightweight backbones such as MobileNetV2 [7] make this practical on laptops and phones. For a demonstration and calibration environment, browser inference has two attractions: it needs no installation, and it keeps the image on the user's device.

Tracking associates detections across frames so that each shopper keeps a consistent identity. Wojke et al. [8] combine motion and appearance cues in a deep association metric for online tracking, and ByteTrack associates almost every detection box, including low-confidence ones, to reduce identity loss under occlusion [10]. Tracks provide the trajectories, velocities and dwell times that this project uses for heatmaps, conversion modelling and loitering detection.

2.4 Retail AI Copilots

Furmakiewicz et al. [1] describe retail copilots, assistants built on language models that help store staff query operational data and act on it. The value of such assistants depends on the quality and freshness of the data they can reach. A copilot that answers questions from a stale or inaccurate inventory record will produce confident but wrong answers.

This observation motivates the design choice in this project to build the grounding layer first: a vision-verified, POS-reconciled view of the shelf that an agent engine can query. The Antigravity agent engine in Layer 3 consumes that view and applies explicit rules and heuristics, so that its recommendations remain explainable.

2.5 Edge-to-Cloud Synchronisation and the Model Context Protocol

Streaming uncompressed video from every camera to the cloud is expensive in bandwidth and raises privacy concerns. An edge-to-cloud design performs inference locally and transmits only the result. Firebase Firestore [12] provides a real-time document database in which clients subscribe to changes, so a telemetry update written at the edge is distributed to every connected dashboard with very low latency.

The Model Context Protocol (MCP) [13] standardises how AI applications call external tools and data sources through typed, schema-described interfaces. Exposing the retail state through an MCP server means that an agent can only perform actions that are declared and validated, which is a useful safety property when the actions include creating purchase

orders. The Firebase MCP server [12] extends this idea to Firebase projects.

2.6 POS Reconciliation and Shrinkage Detection

Shrinkage is the difference between the inventory that should be on hand according to records and the inventory that is actually present. It arises from theft, administrative error, supplier fraud and damage. Detection methods in the literature and in commercial practice generally compare two independent counts of the same quantity and investigate the discrepancy. Exception-based reporting on POS data highlights suspicious voids, refunds and discounts, but it cannot see products that never reach the scanner.

Adding a visual count of shelf depletion provides the second, independent measurement. When the number of items that disappeared from the shelf exceeds the items sold and written off, the remainder is unexplained and deserves attention. Combined with behavioural cues such as rapid extraction followed by movement away from the checkout, this gives a more specific signal than either source alone.

2.7 Existing Systems and Limitations

The literature and the commercial market leave several practical gaps. Shelf-recognition research focuses on detection accuracy and rarely addresses how detections become decisions. Commercial footfall and loss-prevention analytics tend to be separate products with separate data stores. Self-checkout kiosks are commonly treated as payment devices and are not linked to shelf state. Retail copilots depend on underlying records being accurate.

Another gap concerns false alarms. Systems that react to single frames generate alerts whenever a shopper momentarily blocks or removes a product. The cost of false alerts is practical: they consume staff time and reduce trust in the system. Methods for filtering transient events are seldom described in detail in retail-vision papers.

2.8 Research Gap

Based on this review, the gap addressed by this project is the combination of elements that are usually studied or sold separately: real-time shelf and shopper detection, temporal persistence filtering, POS reconciliation with a shrinkage formula, a customer self-checkout that feeds the same inventory state, automated procurement with invoicing, and role-based operator accountability, all inside a single web application that runs on commodity hardware. The project shows how these parts close the loop between raw video pixels, transactions and replenishment.

2.9 Summary

This chapter reviewed work on retail shelf monitoring, real-time detection, browser-based inference, tracking, retail copilots, edge-to-cloud synchronisation and shrinkage detection. Existing methods solve isolated sub-problems and generate false alarms when they react to single frames. These gaps define the research gap addressed by this project. The next chapter presents the system design.

CHAPTER 3
SYSTEM DESIGN

3.1 Existing System

In existing workflows, checking whether a shelf is stocked is a manual, periodic step. A staff member walks the aisle with a handheld scanner, compares what is visible with the ERP record, and corrects the record or places a replenishment request. Between two walks, the shelf may have been empty for a long time without anyone knowing. Loss-prevention staff watch CCTV walls or review footage after an incident, and the POS terminals at checkout report only completed sales. These steps are slow, depend on staff attention, and give no integrated view of the shelf, the shopper and the transaction.

3.2 Proposed System

The proposed system moves observation, reconciliation and replenishment into one continuously running pipeline. Detection runs at the edge, telemetry is synchronised through the cloud layer, the intelligence layer interprets the telemetry over time, and the cockpit presents the result and accepts operator actions. The edge and intelligence stages run continuously, while the transactional actions, such as checkout and restocking, are triggered by customers and managers and write back into the same shared state.

All layers share the same event vocabulary. This design ensures that the camera view, the inventory table, the loss-prevention console and the analytics dashboard cannot disagree with one another, because they all read from, and write to, the same stock and transaction state.

3.3 System Architecture

Figure 3.1 shows the overall architecture of the system. It is read as four layers connected by telemetry. Camera streams, browser webcams and barcode scanners feed Layer 1. The shared event bus and the procurement ledger store the operational state that Layer 4 displays and modifies.

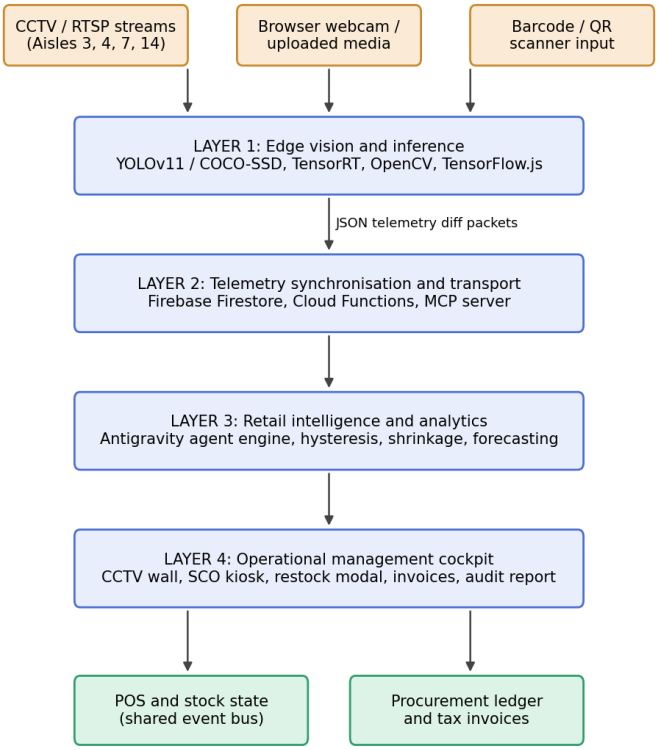


Figure 3.1: System Architecture of the CV-Based Sales Analytics Engine

Layer 1, the edge vision and inference layer, is built with Python, OpenCV, YOLOv11 or COCO-SSD, TensorRT and webcam or RTSP input. Target hardware is a Jetson Orin Nano, a local GPU workstation or browser WebGL. Its role is high-throughput local inference at 30 to 45 frames per second, producing bounding boxes and shopper trajectory tracks without streaming uncompressed video to the cloud.

Layer 2, the telemetry synchronisation and transport layer, uses Firebase Firestore, Cloud Functions and an MCP server. Its role is sub-second synchronisation with a round-trip time under 15 ms, schema-enforced tool execution and bidirectional state distribution. Only JSON telemetry diff packets cross this boundary, which means that a packet describes what changed since the previous one and not the full scene.

Layer 3, the retail intelligence and analytics layer, hosts the Antigravity AI agent engine together with shrinkage heuristics and dwell analysis. Its role is temporal persistence (hysteresis) filtering, loss prevention alerts, dwell-to-purchase conversion modelling and safety-stock forecasting.

Layer 4, the operational management cockpit and presentation layer, is written in vanilla ES6 modules with a custom dark cyberpunk CSS design system and HTML5 Canvas charts.

Its role is to present live CCTV overlays, the smart self-checkout, the restock modal, the tax-invoice archive and the viva defence and daily executive audit dossier.

Every stage reads its input from, and writes its output to, a well-defined interface: packets between Layers 1 and 2, tool calls between Layers 2 and 3, and the event bus between Layers 3 and 4. This keeps each layer replaceable. For example, the browser COCO-SSD detector can be swapped for a YOLOv11 TensorRT service without changing the cockpit.

3.4 Overall Workflow

The workflow for observing and acting on the shelf proceeds in the following order:

1. Camera frames or uploaded media are captured at the edge.
2. The detector identifies products, shelf gaps and shoppers and draws bounding boxes with confidence ratings.
3. Tracks are formed for shoppers, giving trajectories, velocities and dwell times.
4. Compact JSON diff packets describing changed counts and events are sent to the transport layer.
5. The hysteresis filter confirms a stockout only when an item has been absent for the calibrated dwell duration.
6. Visual shelf depletion is reconciled with POS checkout counts and write-offs, and incidents are raised.
7. Safety-stock breaches generate supplier purchase orders, which a manager pays and which produce a signed tax invoice.
8. The cockpit updates the planogram table, camera counts, KPIs and forecasts, and the audit report can be compiled at any time.

3.5 System Modules

Table 3.1 lists the main modules of the implementation and their responsibilities, and Figure 3.2 shows how the application orchestrator connects them.

Module (file)	Responsibility
app.js	Core orchestrator: tab routing and shared event bus.
components/visionFeed.js	Live CCTV wall, YOLOv11 and COCO-SSD detection, overlays, barcode and QR ingest.
components/trainingModal.js	Product calibration and fine-tuning studio using the knowledge base.
components/selfCheckout.js	Smart self-checkout cart, taxation and payment terminal.

Module (file)	Responsibility
components/inventoryAndPos.js	Planogram compliance table, hysteresis buffer and action simulations.
components/lossPrevention.js	Shrinkage anomaly calculation, incident triggers and alarms.
components/analyticsDashboard.js	KPI ribbon, canvas charts and predictive runout forecasting.
components/restockModal.js	Purchase order creation and simulated payment gateway.
components/invoicesTab.js, invoiceModal.js	Procurement ledger and certified tax invoice display.
components/userRoleModal.js, data/authManager.js	Login, registration, RBAC and session handling.
utils/reportGenerator.js	Printable daily store audit dossier.
utils/soundEffects.js	Procedural Web Audio beeps, alarms and cash chimes.
components/vivaGuide.js	Literature survey, research gaps and viva question cards.

Table 3.1: Major System Modules

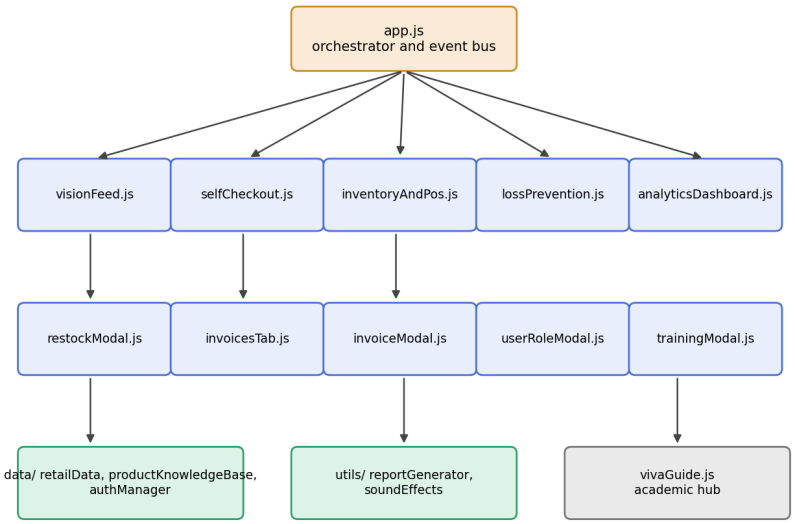


Figure 3.2: Module Map of the Web Application

3.6 Data Flow

Figure 3.3 shows how data moves between the main processes of the system and its persistent stores. Camera and scanner input is turned into telemetry by the detection process. The fusion process reads the telemetry and the POS state and writes the shared inventory and POS state. The analysis process reads that state and raises alerts, and the procurement process turns alerts into purchase orders and invoices that are kept in the invoice ledger. Customers interact through the self-checkout kiosk, which writes purchases back to the same state.

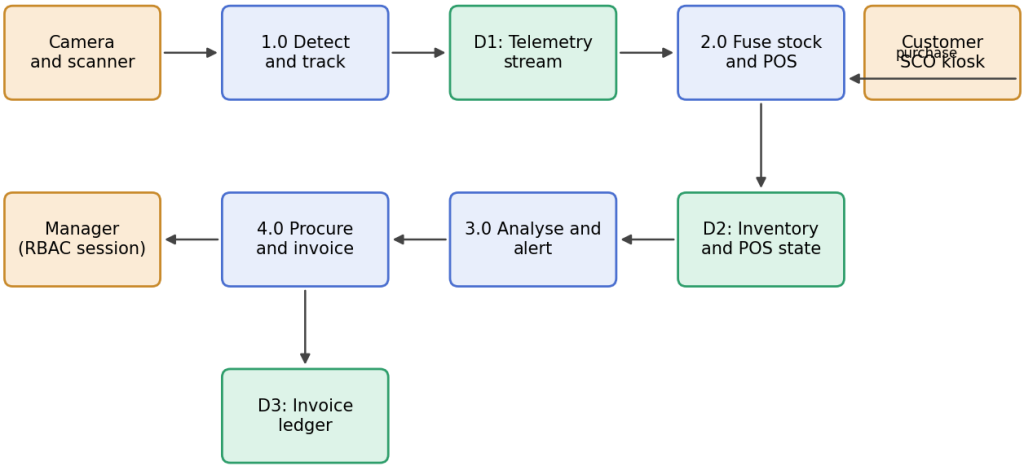


Figure 3.3: Data Flow Between System Processes and Data Stores

The telemetry stream, the inventory and POS state, and the invoice ledger are the only interfaces between the processes. A dashboard component never calls a detector directly; it subscribes to the state. This keeps the user interface responsive and lets each process be developed and tested separately.

3.7 Hardware Requirements

Table 3.2 lists the hardware used for development and deployment targets.

Component	Specification / Purpose
Edge device	NVIDIA Jetson Orin Nano or a local GPU workstation for YOLOv11 TensorRT inference at 30 to 45 FPS.
Cameras	Commodity RTSP CCTV cameras and a standard USB or laptop webcam.
Client device	Any computer or tablet with a modern browser supporting ES6 modules, WebGL and Web Audio.
Kiosk terminal	Touch-screen display with a camera and optional barcode scanner for the self-checkout.
Network	Local network for RTSP feeds and internet connectivity for Firebase synchronisation.

Table 3.2: Hardware Requirements

Inference is performed on the edge device so that only small telemetry packets use the network. A single browser is sufficient for demonstration, because TensorFlow.js executes COCO-SSD on the client GPU.

3.8 Software Requirements

Table 3.3 lists the main software components.

Component	Purpose
Python, OpenCV	Edge capture, pre-processing and RTSP handling.
YOLOv11 (Ultralytics) [9], TensorRT	High-throughput object detection at the edge.
TensorFlow.js [6], COCO-SSD	Client-side neural inference in the browser.
Firebase Firestore, Cloud Functions [12]	Real-time telemetry synchronisation and server logic.
Model Context Protocol server [13]	Schema-enforced tool execution for the agent engine.
Antigravity AI agent engine	Shrinkage heuristics, dwell analysis and forecasting.
Vanilla JavaScript (ES6 modules)	Application logic with no external bundler.
HTML5 Canvas, custom CSS	60 FPS charts and the dark glassmorphic design system.
Web Audio API	Procedural sound effects without audio files.
Browser LocalStorage	Operator session persistence.

Table 3.3: Software Requirements

3.9 Non-Functional Design Considerations

Beyond the functional modules, the design is shaped by a small set of non-functional requirements. They explain several of the choices made in the previous sections and are summarised in Table 3.4.

Requirement	Design response
Latency	Local edge inference; diff packets; round-trip target under 15 ms for synchronisation.
Privacy	Raw video stays at the edge or on the client device; only telemetry crosses the network.
Bandwidth	Packets describe changes only, so many cameras can share one connection.
Explainability	Rule-based hysteresis, an explicit shrinkage formula and logged incident triggers.
Accountability	RBAC with every event tagged by the active operator identity.
Portability	Vanilla ES6 modules, no bundler; runs in any modern browser from a static server.
Maintainability	One module per responsibility; shared event bus; reference data kept in the data folder.
Usability	Single-page cockpit, quad-wall view, audible feedback and a one-click audit report.

Table 3.4: Non-Functional Requirements and Design Responses

Privacy deserves particular attention in a system that tracks shoppers. The design minimises exposure by extracting trajectories, velocities and dwell times at the edge and sharing only those derived values. Shopper tracks are used for aggregate heatmaps, conversion modelling

and loitering alerts, and they are not tied to a personal identity. Any production deployment would also need signage, a retention policy and compliance with the data-protection rules that apply in its jurisdiction.

Resilience is addressed by separating the layers. If the cloud connection is lost, edge detection and local alarms continue to run, and diff packets can be queued and sent when the link returns. If a camera feed is interrupted, the corresponding channel shows an offline state and the camera uptime figure in the audit report reflects it, while the other channels are not affected.

3.10 Summary

This chapter described the existing and proposed systems, the four-layer architecture, workflow, modules and data flow of the CV-Based Sales Analytics Engine, and the hardware and software it needs. The design keeps perception, transport, intelligence and presentation separate, so each layer can be replaced independently. The next chapter explains how each module was implemented.

CHAPTER 4
IMPLEMENTATION AND METHODOLOGY

4.1 Runtime Environment and Application Orchestration

The application is written in pure vanilla JavaScript using ES6 modules and runs natively in modern browsers with no external bundler dependencies. A single page, index.html, hosts the dashboard, and test.html mirrors it for testing. The master stylesheet, styles.css, implements the dark cyberpunk and glassmorphic design system with glowing telemetry indicators, and all charts are drawn on high-DPI HTML5 Canvas elements.

The file app.js is the core orchestrator. It performs tab routing, mounts each component module into its panel, and provides a shared event bus. Components do not call one another directly. Instead, a component publishes an event such as a checkout, a pickup, a restock or a login, and every interested component subscribes to it. This loose coupling is what allows a self-checkout purchase to update the inventory table, the analytics dashboard and the telemetry drawer at the same time.

A fixed, collapsible telemetry drawer at the bottom of the screen shows the live packet stream across edge, cloud, POS and authentication events. Each packet carries a timestamp, a source layer, an event type and the identity of the active operator. The drawer serves both as an operational log and as a demonstration aid, since it makes the invisible flow of data between the layers visible.

4.2 Live Edge CCTV and Vision Feed Engine

The component components/visionFeed.js implements the vision feed. Four camera channels are configured, and Table 4.1 lists them together with the products and behaviours each channel is responsible for. Static test images in the assets folder (camera_perfume.jpg, camera_dairy.jpg, camera_cereal.jpg and camera_traffic.jpg) stand in for the live streams when no camera is attached.

Camera	Location	Content and detection focus
CAM 02	Aisle 3	Luxury fragrances and perfumes: Chanel No. 5, Dior Sauvage Elixir, skincare serums.
CAM 04	Aisle 4	Beverages, milk jugs, oat milk, biscuits and preserved pickles.
CAM 07	Aisle 7	Dry groceries, cereals, chips, shampoos and active out-of-stock shelf gaps.
CAM 14	Overhead concourse	De-warped fisheye feed tracking shopper trajectories, velocities and dwell times.

Table 4.1: Configured CCTV Channels

Direct browser detection integrates TensorFlow.js and COCO-SSD. A manager can connect a webcam or upload a video or image file, and the model runs entirely in the browser, which keeps the image on the local device. For production-grade accuracy, hooks for YOLOv11 TensorRT backends and RTSP streams are provided, so that the same interface can display detections produced by an edge device.

The engine supports two viewing modes. Focused single-camera view maximises one channel for close inspection, and quad-wall mode shows four synchronised channels as a surveillance wall. Visual overlays include interactive bounding boxes with confidence ratings, planogram alignment guidelines, dwell heatmaps and shopper path motion trails. The engine also ingests one-dimensional and two-dimensional barcodes and QR codes for item verification, which gives a second, optical path to confirm an item recognised visually.

4.3 AI Product Training and Calibration Studio

The studio, implemented in components/trainingModal.js with data in data/productKnowledgeBase.js, lets a manager train, fine-tune and calibrate recognition weights for specific products without hardcoded assumptions. The knowledge base stores, for each product kind, the visual aspect ratio, the packaging material and the heuristics that distinguish it from neighbouring classes. Table 4.2 lists the supported taxonomy.

Class	Visual cue used for recognition
Beverages	Cylindrical aspect ratio and metallic, specular aluminium reflection.
Chips and snacks	Nitrogen-flushed, metallised foil pillow pouches.
Pickles and condiments	Heavy glass jars with airtight vacuum lug caps.
Biscuits and cookies	Rectangular foil trays.
Shampoos and personal care	Ergonomic pump dispenser bottles.
Perfumes and fragrances	Faceted glass flacons with atomisers.
Cosmetics	UV dropper pipette bottles and serums.
Dairy milk	HDPE plastic jugs with moulded handles.
Breakfast cereals	Recyclable paperboard cartons.
Human shoppers	Pedestrian tracking and dwell monitoring.

Table 4.2: Supported Product Taxonomy and Visual Cues

Separating the taxonomy from the code has a practical benefit. When a new product line arrives, the manager adds or adjusts its entry in the studio and the recognition behaviour follows, without a code change. It also makes explicit why a class is difficult: reflective cans and glass flacons, for instance, are prone to specular highlights that confuse detectors, and the knowledge base records the cues that help overcome this.

4.4 Shelf Stock and POS Fusion with Temporal Persistence

The component components/inventoryAndPos.js fuses the visual shelf count with POS velocity. Its planogram compliance table shows, for every SKU, the SKU identifier, shelf coordinates, visual stock count, safety buffer threshold and daily POS velocity. Two action buttons, Simulate Customer Pickup and Restock Supplier Order, trigger live tests of the full loop.

A single frame in which a product is missing does not prove a stockout. The temporal persistence buffer, a hysteresis filter, therefore requires the item to remain absent for a calibrated dwell duration before a critical replenishment alert is raised. Figure 4.1 shows the states. A product starts In Stock. When the detector stops seeing it, the state moves to Candidate Gap and a timer starts. If the item returns before the timer expires, the event is classed as a pickup and the state returns to In Stock. If the absence lasts for the dwell duration T_dwell, the state becomes Confirmed Stockout and a restock alert is produced.

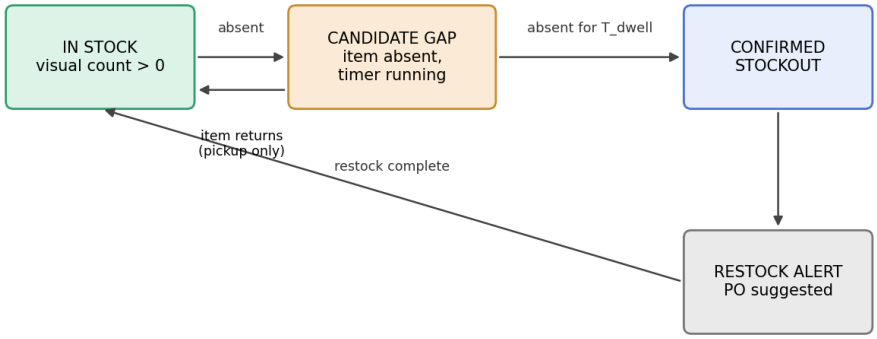


Figure 4.1: Hysteresis (Temporal Persistence) State Machine for a Shelf Facing

The rule can be written compactly. Let a(t) be 1 when the SKU is absent in the frame at time t and 0 otherwise, and let t0 be the time at which the current run of absence began:

Stockout(t) = 1 if a(s) = 1 for all s in [t0, t0 + T_dwell], otherwise 0 (1)

Equation (1) means that a stockout is declared only after a continuous run of absence that lasts at least T_dwell. Choosing T_dwell is a trade-off. A short value reacts quickly but admits false alarms from shoppers who are reading a label, while a long value is robust against pickups but delays the replenishment of a genuinely empty shelf. For this reason the dwell duration is a calibrated, per-category setting and not a global constant.

4.5 CCTV Loss Prevention and Anti-Theft Surveillance

The component components/lossPrevention.js converts the discrepancy between what leaves the shelf and what is sold into a quantitative signal. The mathematical shrinkage anomaly is calculated as

Shrinkage Delta = (Visual Shelf Depletion) - (POS Checkout Count + Documented Write-offs) (2)

where visual shelf depletion is the number of items that the vision layer has seen disappear from the shelf, the POS checkout count is the number of units registered at the kiosk or terminal, and documented write-offs are items formally removed for damage or expiry. A positive delta means that more items left the shelf than can be explained by sales and write-offs. Figure 4.2 shows the decision logic.

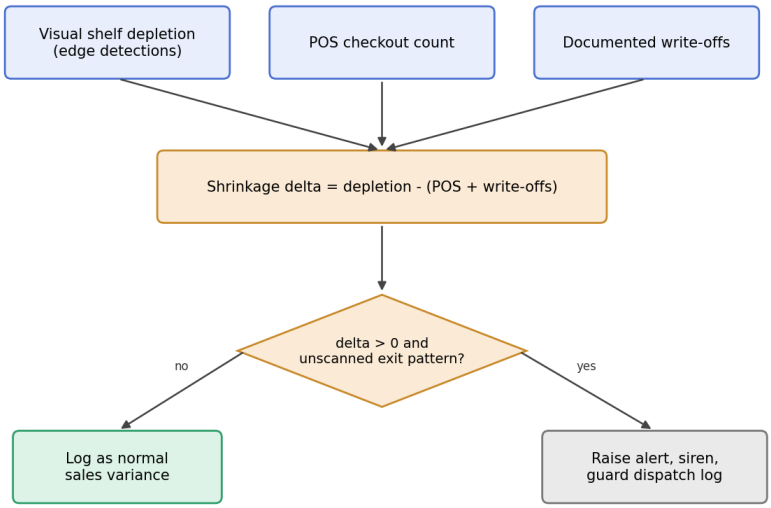


Figure 4.2: Shrinkage Anomaly Calculation and Decision Logic

Three incident triggers use behaviour as well as the numerical delta, so that a positive delta alone does not cause an alarm:

- Unscanned concealment: rapid item extraction from the shelf followed by movement away from the checkout without a POS registration.
- Planogram breaches: a misplaced cross-category item, for example a non-edible shampoo placed on a cereal shelf.
- Loitering and dwell anomalies: a shopper who remains stationary in a blind spot for more than 4.5 minutes.

Security responses are an acoustic alarm siren synthesised by utils/soundEffects.js and a security-guard dispatch action that is written to the incident log together with the identity of the operator who issued it. The alarm and the log entry are separate so that a manager can acknowledge an alert without dispatching a guard, and the choice is recorded either way.

4.6 Customer Smart Self-Checkout Kiosk

The component components/selfCheckout.js implements the customer-facing kiosk. Its vision-to-cart pipeline ingests items that the camera has identified or that have been optically scanned directly into a billing cart. Figure 4.3 shows the flow from item presentation to stock update.

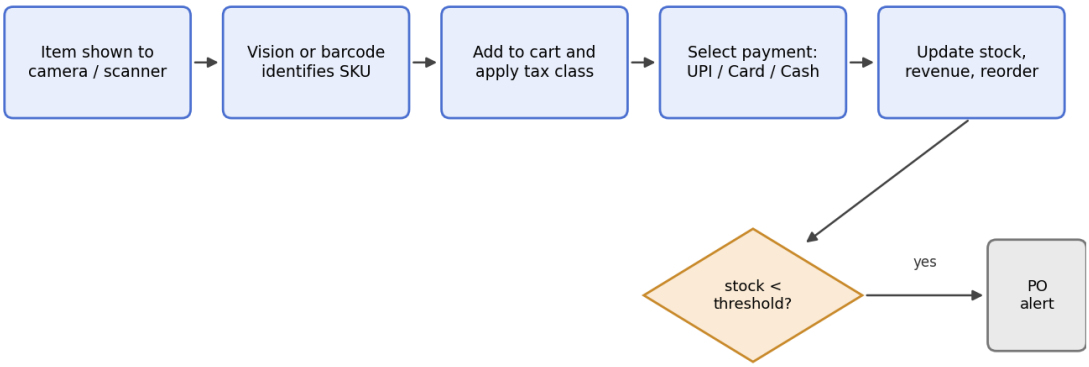


Figure 4.3: Self-Checkout Flow from Item Recognition to Stock Update

Taxation depends on product class. Staple edible foods are taxed at 5%, whereas luxury fragrances and cosmetics are taxed at 18%. For a cart line with unit price p, quantity q and tax rate r, the line total is

$$\textit{Line total} = p \times q \times (1 + r) \quad (3)$$

and the bill is the sum of the line totals. The kiosk supports mock payment flows via UPI (Google Pay or PhonePe QR code), credit or debit card, and cash. Each payment mode produces its own confirmation sequence and sound cue.

The kiosk performs closed-loop reconciliation. Every completed checkout immediately updates the global shelf stock levels, triggers a low-stock reorder threshold if one is breached, and increments the real-time store revenue metric. Because the inventory table, the analytics dashboard and the loss-prevention console read the same state, the effect of a purchase is visible everywhere without any manual synchronisation.

4.7 Store Analytics, Spatial Heatmaps and Forecasting

The component components/analyticsDashboard.js provides the executive view. A metrics ribbon shows total footfall, POS store revenue, conversion yield and average aisle dwell time. For conversion, shoppers with a dwell time of at least 2.5 minutes exhibit an 84% purchase conversion, which gives managers a measurable link between time spent in an aisle and sales.

An interactive 60 FPS canvas chart compares hourly visitor footfall, POS sales volume and conversion velocity. Predictive runout forecasting estimates the remaining operating runway before a stockout from the current stock and the current sales velocity:

$$\textit{Runway (hours)} = \textit{Current stock} / \textit{Sales velocity (units per hour)} \quad (4)$$

When the runway falls below the supplier lead time, the product is a candidate for replenishment before the safety buffer is breached. This turns the dashboard from a record of the past into a planning tool. Spatial heatmaps built from the overhead concourse camera show where shoppers dwell, which helps a manager judge whether a promotional display attracts attention.

4.8 Procurement Restock and Secure Payment Gateway

The component components/restockModal.js generates a supplier purchase order when a SKU breaches its minimum safety level. Figure 4.4 shows the sequence. The manager reviews the order, selects a payment channel, and the modal simulates authorisation through credit or debit card, Google Pay or UPI instant VPA, or corporate net banking.



Figure 4.4: Procurement Workflow from Safety-Stock Breach to Updated Counts

On success the modal emits a signed digital tax invoice with a verification stamp and updates the live camera detection counts, so that the vision layer reports the restocked quantity. Closing the loop in this way means the next detection cycle confirms that the delivery has reached the shelf.

4.9 Tax Invoices and Procurement Records Archive

Two components, components/invoicesTab.js and components/invoiceModal.js, maintain the ledger. The archive tab lists all procurement invoices and supplier transactions in one place. Selecting an entry opens a full-featured modal that displays line items, the supplier tax identification, the authorised manager identifier and a print action. Because each invoice carries the operator identity that approved it, the ledger doubles as an audit trail.

4.10 User Role Management and Access Control

The modules data/authManager.js and components/userRoleModal.js implement role-based access control. Operator profiles harish_manager and admin are pre-seeded, and the modal supports new user registration and credential validation. Sessions are saved to LocalStorage so that a refresh does not log the operator out. Figure 4.5 shows the sequence.



Figure 4.5: Authentication and Role-Based Access Flow

The telemetry stream automatically tags each event with the identity of the active operator. A restock, a guard dispatch and a report generation can all be attributed to a named person. In a production deployment, LocalStorage would be replaced by server-side session management, but the interface of the module would remain the same.

4.11 Daily Store Audit and Intelligence Report Generator

The utility utils/reportGenerator.js compiles a publication-ready store audit dossier in one click. The dossier combines camera uptime, planogram compliance scores, inventory gaps and procurement spend, and is formatted for printing or saving as a PDF. Managers can therefore hand a consistent summary to head office at the end of every day without assembling figures from several screens.

4.12 Academic Review 2 Viva Defence Hub

The component components/vivaGuide.js is the academic companion of the system. It contains a literature survey that reviews Furmakiewicz et al. on retail AI copilots (2024), Glenn Jocher and Ultralytics on YOLO11 (2024), and the Google Firebase MCP server (2026). It lists the identified research gaps, showing how prior work solves isolated sub-problems whereas this system closes the loop between raw video pixels, POS transactions and automated procurement. It also contains viva question cards with technical justifications for model selection, occlusion mitigation, hysteresis filters and MCP security safeguards.

4.13 Audio Synthesiser and Telemetry Drawer

The utility utils/soundEffects.js is a fully procedural Web Audio API [11] engine. It generates clicks, scan beeps, payment confirmations, cash chimes and security alarms from oscillators and gain envelopes, with no external audio files. Avoiding audio assets keeps the application small, and it removes any licensing question about sound files. Audible feedback matters in this setting because a manager watching a CCTV wall cannot always look at the alert panel.

4.14 Reproducibility and Data Management

Reference data live in the data folder. The file retailData.js holds the camera definitions, the inventory catalogue, the hourly analytics series and the viva data. The file productKnowledgeBase.js holds the product taxonomy, aspect ratios and packaging heuristics, and authManager.js holds the credentials store and active session. Because the application uses no bundler, the project can be opened by serving the folder with any static web server, and the exact files that were demonstrated are the files that run.

The code, assets and documentation are organised by responsibility into components, data, utils and assets folders. This structure is described in Section 9.2 and keeps the cross-cutting utilities independent of the user interface components.

4.15 Detection Post-Processing, Dwell and Heatmap Computation

Between the raw detector output and the telemetry packet there is a short post-processing stage. The detector returns candidate boxes with a class and a confidence. Boxes below a confidence threshold are discarded, and overlapping boxes of the same class are merged by non-maximum suppression so that one product is not counted twice. The surviving boxes are matched to the planogram: a box whose centre falls inside the coordinates of a facing is counted for that facing, and an empty facing with no matching box becomes a candidate gap.

Planogram alignment uses the overlap between a detected box and the expected facing. For a detected box B and a facing region F, the intersection over union is

$$IoU(B, F) = Area(B \text{ and } F) / Area(B \text{ or } F) \quad (5)$$

and the box is assigned to the facing for which the IoU is largest, provided it exceeds a minimum overlap. A box with a high confidence whose best-matching facing belongs to a different category is the evidence used for the planogram-breach trigger of Section 4.5.

Dwell time is computed from the tracks of the overhead camera. If a shopper track enters an aisle zone at time t_in and leaves it at time t_out, the dwell is t_out - t_in, and the aisle

average is the mean over all tracks that visited the zone in the period. A track whose position stays within a small radius for longer than the loitering threshold is flagged. The same position samples are accumulated into a grid over the floor plan, and the grid values are drawn as a colour gradient to form the spatial dwell heatmap.

Finally, the post-processed result is compared with the previous result for the same camera, and only the differences are written to the telemetry packet. If nothing changed, nothing is sent. This difference encoding is what keeps the volume of telemetry small even when many cameras are active.

4.16 Summary

This chapter described the implementation of each module: the orchestrator and event bus, the vision feed, the calibration studio, the hysteresis-based stock and POS fusion, the loss-prevention formula, the self-checkout kiosk, the analytics and forecasting dashboard, procurement and invoicing, access control, the audit report, the viva hub and the audio engine. The next chapter reports how the system behaves.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Experimental Setup

The system was exercised as a web application in a modern desktop browser. Detection was tested with the four static CCTV assets and with a connected webcam, and the transactional modules were driven through the built-in simulation actions. Quantitative figures in this chapter that describe throughput, latency and conversion are the design targets and analytical results stated in the project specification, and worked examples are labelled as such. The chapter does not present benchmarking results for hardware that was not part of the demonstration.

5.2 Choice of Detection Backend

The system supports two detection backends, and they are intended for different roles. Table 5.1 compares them. COCO-SSD running in TensorFlow.js is chosen for the interactive demonstration and calibration environment because it needs no installation and keeps data on the client. YOLOv11 on a TensorRT-accelerated edge device is chosen for continuous production monitoring because it reaches the target frame rate on embedded hardware.

Aspect	COCO-SSD (browser)	YOLOv11 (edge)
Runtime	TensorFlow.js with WebGL	TensorRT on Jetson Orin Nano or GPU workstation
Input	Webcam, uploaded video or image	RTSP CCTV streams and webcams
Installation	None, runs in the page	Python, OpenCV and the model runtime
Target throughput	Interactive demonstration	30 to 45 FPS
Data location	Image stays on the device	Inference at the edge, only telemetry leaves
Primary role	Demonstration and calibration	Continuous store monitoring

Table 5.1: Comparison of the Two Detection Backends

Keeping both behind the same interface is a design advantage. The cockpit consumes the same bounding-box and confidence structure from either source, so a store can start with browser detection and move to edge inference later without changing the screens or the stored data.

5.3 Vision Feed Results

On the four camera channels the vision engine drew bounding boxes with confidence ratings, planogram alignment guidelines and, on the overhead channel, shopper trails and dwell heatmaps. The quad-wall mode displayed the four channels simultaneously, and switching between focused and wall view did not interrupt the telemetry drawer. Barcode and QR ingest provided a second path for verification when the visual recognition of a product was uncertain.

The Aisle 7 channel is the one on which shelf-gap detection is most visible, because it contains active out-of-stock gaps. Aisle 3 shows why the knowledge base matters: faceted glass flacons produce specular highlights, and the recognition cues recorded for the perfume class are used to separate them from neighbouring skincare serums. The overhead concourse channel demonstrates the largest benefit of tracking, since dwell and trajectory are properties of a shopper over time and cannot be read from a single frame.

5.4 Hysteresis Filter Verification

The behaviour of the temporal persistence buffer can be explained with two cases, shown in Figure 5.1. In case A a shopper lifts a product, inspects it and returns it. The absence signal is raised for a short interval and ends before T_dwell, so the filter classes it as a pickup and raises no alert. In case B the product disappears and does not return. The absence signal persists past T_dwell and the filter confirms the stockout and proposes replenishment.

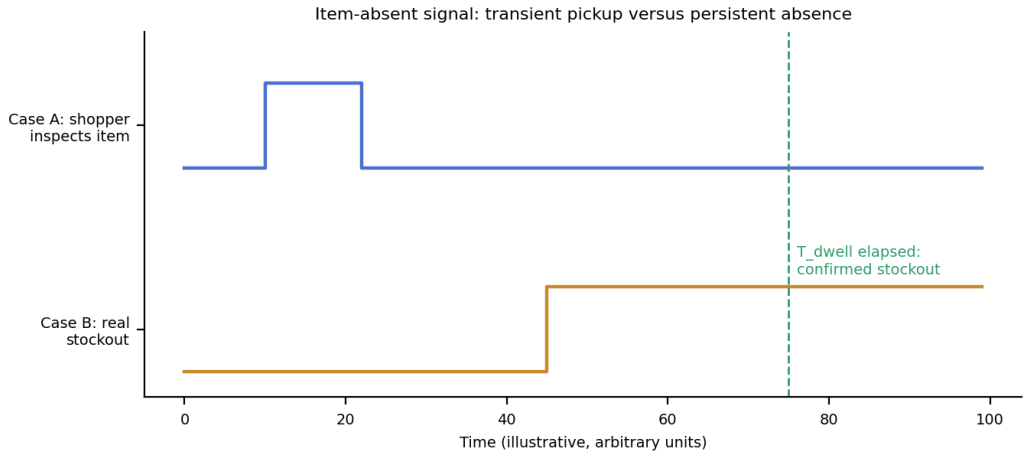


Figure 5.1: Item-Absent Signal: Transient Pickup versus Persistent Absence (Illustrative)

The Simulate Customer Pickup action in the inventory module reproduces case A in the demonstration, and a stock count that reaches zero reproduces case B. The result is that the number of critical alerts reflects genuine gaps and not shopper curiosity, which is the property that keeps staff confidence in the system.

5.5 Shrinkage Anomaly Worked Example

Equation (2) is easiest to understand with numbers. Table 5.2 gives an illustrative example for one SKU over an observation window. The values are chosen to show the arithmetic and are not measurements.

Quantity	Value (units)
Visual shelf depletion	12
POS checkout count	9
Documented write-offs	1
Shrinkage delta = 12 - (9 + 1)	2

Table 5.2: Illustrative Shrinkage Calculation for One SKU

The delta of two units is unexplained. If the behavioural trigger for unscanned concealment has also fired for the same SKU and aisle, the system raises an incident. If it has not, the discrepancy is logged as a variance for review, since small differences can come from detection errors or items temporarily relocated by shoppers. Combining the numerical and behavioural evidence is what keeps the false-alarm rate manageable.

5.6 Conversion and Dwell Analysis

The analytics module reports that shoppers whose dwell time is at least 2.5 minutes show 84% purchase conversion. This is a useful operating threshold. A manager can see that an aisle with high footfall but low dwell is not converting, and can investigate signage, shelf layout or stock availability. Combined with the hysteresis filter the analysis avoids a common mistake: a low-selling product may be unavailable and not unpopular.

The loitering rule applies the same dwell measurement from a security viewpoint. A shopper who is stationary in a blind spot for more than 4.5 minutes is flagged, which is well beyond the dwell associated with ordinary browsing. The two thresholds, 2.5 and 4.5 minutes, therefore separate engaged shopping from behaviour that deserves a closer look.

5.7 Runout Forecast and Tax Worked Examples

Equation (4) is illustrated by a product with 36 units on the shelf and a current sales velocity of 6 units per hour. Its runway is 36 / 6 = 6 hours. If the supplier lead time is longer than 6 hours, the product should be ordered now, before the safety buffer is breached.

Equation (3) is illustrated with two lines on a self-checkout bill. A staple food item priced at 200 with quantity 1 is taxed at 5%, giving 210. A fragrance priced at 1,000 with quantity 1 is taxed at 18%, giving 1,180. The bill is 1,390. These values are for explanation only.

5.8 Closed-Loop Behaviour

The most important result is not any single module but the fact that their effects connect. When a purchase is completed at the kiosk, the stock count in the inventory table decreases, the revenue counter on the analytics dashboard increases, and, if the safety buffer is breached, a reorder is proposed. When the manager completes the restock payment, a signed tax invoice appears in the archive and the camera detection count for the SKU increases. Every one of these steps appears in the telemetry drawer with the operator identity, which makes the chain verifiable end to end.

5.9 Computational Performance

The architecture is designed so that the expensive work is done once, close to the camera, and the cheap work is distributed. Edge inference targets 30 to 45 frames per second on a Jetson Orin Nano or GPU workstation. The synchronisation layer targets a round-trip time below 15 ms and sends only diff packets, so bandwidth use is small compared with streaming video. The dashboard charts are drawn on canvas at 60 frames per second.

These are design targets. Their verification on the production hardware is part of the planned work in Chapter 7. In the browser demonstration, the limiting factor is the client GPU running COCO-SSD, and the remaining modules, which perform only light arithmetic and DOM updates, add negligible cost.

5.10 Functional Testing

Table 5.3 lists the functional checks defined for the system, together with the behaviour expected of each. They were designed to be exercised manually through the interface, and an automated test suite is planned as future work.

Test area	Check performed	Expected outcome
Vision feed	Open each of the four channels and the quad wall	Boxes, confidence and overlays render; no feed blocks another
Browser detection	Connect a webcam or upload an image	COCO-SSD returns labelled boxes in the page
Hysteresis	Simulate a pickup, then a persistent absence	Pickup raises no alert; persistent absence confirms a stockout
Shrinkage	Create depletion with no POS entry	Positive delta and an incident entry with alarm
Self-checkout	Add items of 5% and 18% classes and pay	Correct tax, stock decremented, revenue incremented
Restock	Pay a purchase order by each channel	Invoice issued, camera counts updated
Invoice archive	Open an invoice and print	Line items, supplier ID and manager ID displayed
Access control	Log in, register, refresh the page	Session persists; events tagged with the operator
Audit report	Generate the daily dossier	Printable report with uptime, compliance, gaps and spend

Table 5.3: Functional Testing Checklist

Each check corresponds to a requirement in Table 1.1. The actual outcomes should be recorded against this checklist at the time of the review demonstration.

5.11 Overall Discussion

The central finding of the project is that perception, transaction and procurement can be joined in one browser-native application with no bundler and with modest hardware. The hysteresis filter addresses a practical problem, the false alarm, that is often glossed over in vision research. The shrinkage formula gives a simple and explainable measure, and the incident triggers add the behavioural context that a number alone lacks.

The edge-to-cloud design is another important decision. By sending only telemetry diffs, the system preserves shopper privacy, reduces bandwidth and allows many cameras to share one connection. The MCP server adds a controlled path for agents, so that an automated action such as raising a purchase order must pass through a declared and validated tool.

The system is best used as an operational assistant. Its alerts are risk signals that a person confirms. A positive shrinkage delta does not prove theft, and a confirmed stockout depends on the detector seeing the shelf correctly. Occlusion, lighting and camera placement all matter, and the knowledge base and calibration studio exist so that these can be tuned for each store.

5.12 Comparison with Existing Work

The proposed system is compared with the existing approaches of Section 1.3 and Chapter 2 in Table 5.4. The comparison is based on the capabilities each approach offers, because the approaches use different data and no head-to-head accuracy experiment was run in this project.

Approach	Sees the shelf	Links to POS	Filters transient events	Automates restock
Manual barcode audit	Periodic	No	Not applicable	No
POS and ERP records only	No	Yes	Not applicable	Partly
Detection-only research systems	Yes	No	Rarely	No
Separate commercial analytics	Yes	Limited	Varies	No
Retail copilot [1]	Depends on data	Depends on data	No	Advisory
This work	Yes, continuous	Yes	Yes, hysteresis	Yes, PO and invoice

Table 5.4: Comparison of the CV-Based Sales Analytics Engine with Existing Approaches

The advantage of the integrated approach is that no part of the loop is left to manual work. Its limitations are that it depends on camera coverage and detection quality, and that payment and invoicing are simulated in the present version.

5.13 Threats to Validity

Several factors limit how far the results of this chapter can be generalised. First, the vision demonstrations use four static CCTV images and a webcam, which cannot represent the variety of lighting, camera angles and crowding found across a real store. Detection accuracy on cluttered shelves with heavy occlusion may be lower than on the demonstration images.

Second, the throughput, latency and conversion figures are design targets and analytical statements from the project specification. They indicate what the architecture is intended to achieve, and they need to be confirmed by measurements on the target hardware. Third, the worked examples for shrinkage, forecasting and taxation are illustrative, and the thresholds used by the hysteresis and loitering rules need calibration against real shopper behaviour in the store where the system is deployed.

Fourth, the payment and invoicing flows are simulated, so the results say nothing about the behaviour of real payment gateways, settlement delays or statutory invoicing rules. These

limits are the reason the alerts are framed as risk signals and the reason the future scope in Chapter 7 begins with hardware benchmarking and store-specific fine-tuning.

5.14 Summary

This chapter described the detection backends, the behaviour of the vision feed, the hysteresis filter, the shrinkage, forecasting and tax calculations through worked examples, the closed-loop behaviour, the performance targets, the functional checklist and a comparison with existing work. The integrated system connects pixels, transactions and procurement, and its alerts are designed to be explainable and reviewable by a person.

CHAPTER 6
DASHBOARD WALKTHROUGH AND DEMONSTRATION

6.1 Overview

This chapter describes how the operations cockpit is used and what each screen shows. Section 6.2 explains the navigation and the telemetry drawer. Section 6.3 walks through six demonstration scenarios that exercise the full loop, using the schematic screens in Figures 6.1 to 6.6. Section 6.4 shows the structure of a telemetry packet. The modules themselves are described in Chapter 4, and the checklist used to verify them is given in Chapter 5.

6.2 Cockpit Navigation and Telemetry Drawer

The cockpit is a single page with a tab for each major function: live CCTV, inventory and POS, loss prevention, analytics, self-checkout, invoices and the viva hub. The tab router in app.js shows one panel at a time while all modules stay subscribed to the shared event bus, so a change made in one tab is already present when the manager opens another.

The active operator is shown in the header, and a role modal handles login and registration. The pre-seeded profiles harish_manager and admin allow the system to be demonstrated immediately. At the bottom of the screen, the collapsible telemetry drawer lists packets as they occur, labelled by origin: edge, cloud, POS or authentication. The drawer is the quickest way to confirm that a given action has reached the shared state.

6.3 Demonstration Scenarios

Scenario A: live surveillance. The manager opens the CCTV tab and selects the quad-wall mode, shown schematically in Figure 6.1. Four channels appear together: Aisle 3, Aisle 4, Aisle 7 and the overhead concourse. Selecting a single channel enlarges it, and the overlays show boxes, confidences, planogram guidelines, dwell heatmaps and shopper trails. A webcam or an uploaded image can be passed to the in-browser COCO-SSD detector to show live detection.

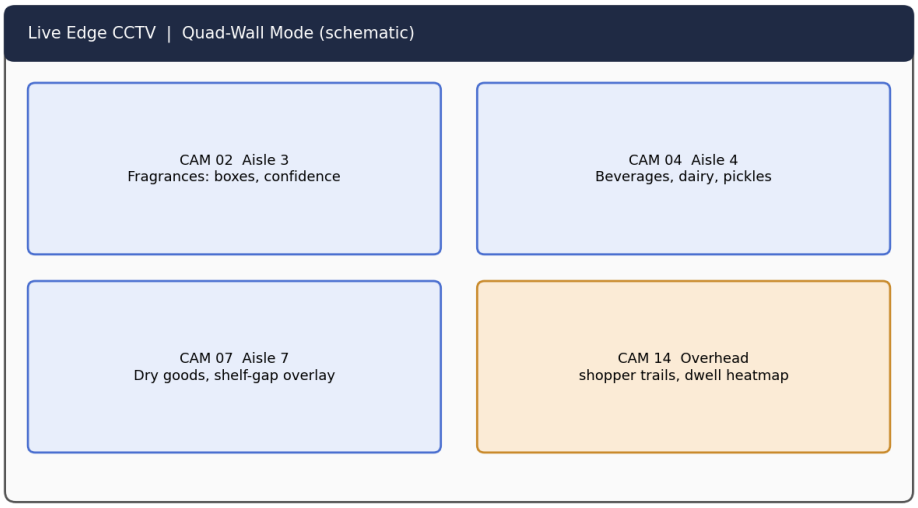


Figure 6.1: Schematic of the Live CCTV Quad-Wall Screen

Scenario B: phantom stockout versus pickup. In the inventory tab, shown schematically in Figure 6.2, the planogram table lists each SKU with its shelf coordinates, visual count, safety buffer and POS velocity. Pressing Simulate Customer Pickup briefly lowers a count, and the hysteresis badge shows a candidate gap that clears when the item returns. Reducing a count to zero and leaving it there turns the badge to a confirmed stockout after the dwell duration, and a restock suggestion appears.

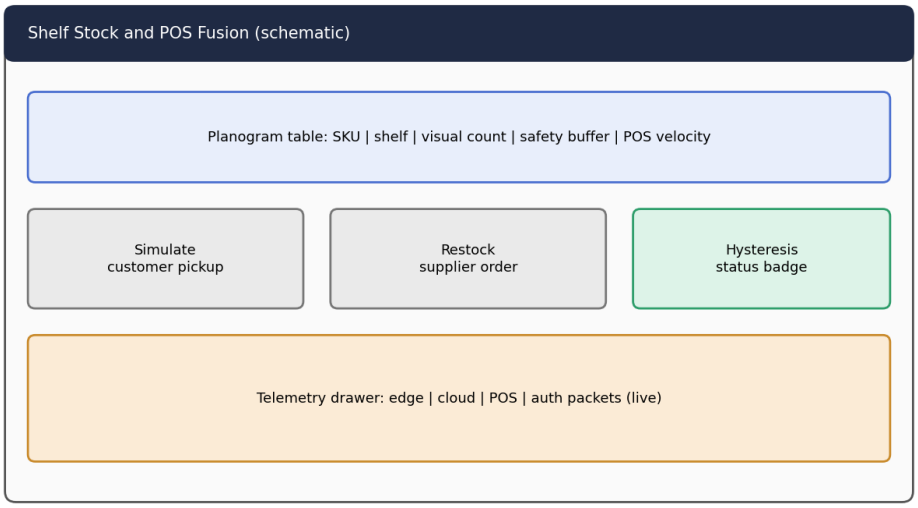


Figure 6.2: Schematic of the Shelf Stock and POS Fusion Screen

Scenario C: self-checkout. At the kiosk, shown schematically in Figure 6.3, the customer presents an item to the camera or scanner. The recognised item is added to the cart with the correct tax class, and the customer pays by UPI QR, card or cash. A confirmation sound plays, the stock count falls, and the revenue figure on the analytics screen rises.

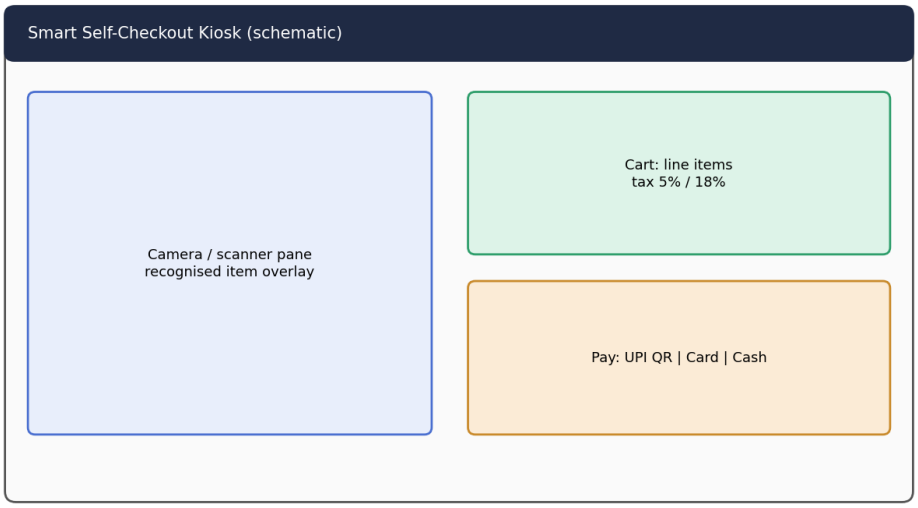


Figure 6.3: Schematic of the Smart Self-Checkout Kiosk

Scenario D: loss prevention. In the loss-prevention tab, shown schematically in Figure 6.4, an unscanned concealment event, a planogram breach or a loitering alert appears in the incident feed. The shrinkage delta for the SKU is shown beside the feed. The manager can sound the siren and log a guard-dispatch action, and both appear in the telemetry drawer under the manager's identity.

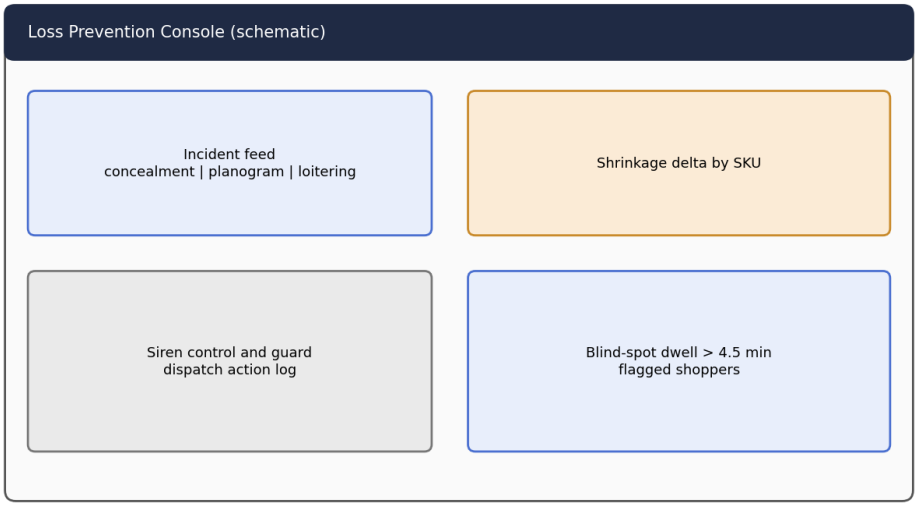


Figure 6.4: Schematic of the Loss Prevention Console

Scenario E: analytics and forecasting. The analytics tab, shown schematically in Figure 6.5, opens with the KPI ribbon. The hourly canvas chart compares footfall, sales and conversion, and the runout panel lists the remaining hours for each SKU, with the shortest runway first.

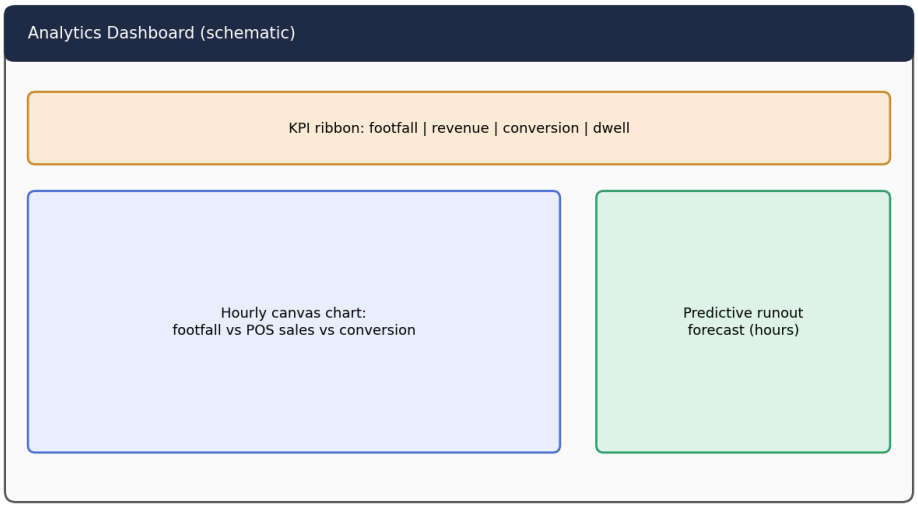


Figure 6.5: Schematic of the Analytics Dashboard

Scenario F: restocking and invoicing. When a SKU needs replenishment, the manager opens the restock modal, shown schematically in Figure 6.6, reviews the purchase order and pays through the chosen channel. A signed invoice with a verification stamp is produced and stored in the archive, where it can be opened and printed. The camera counts update to reflect the delivery. Finally, the daily audit dossier can be generated in one click, summarising uptime, planogram compliance, gaps and procurement spend.

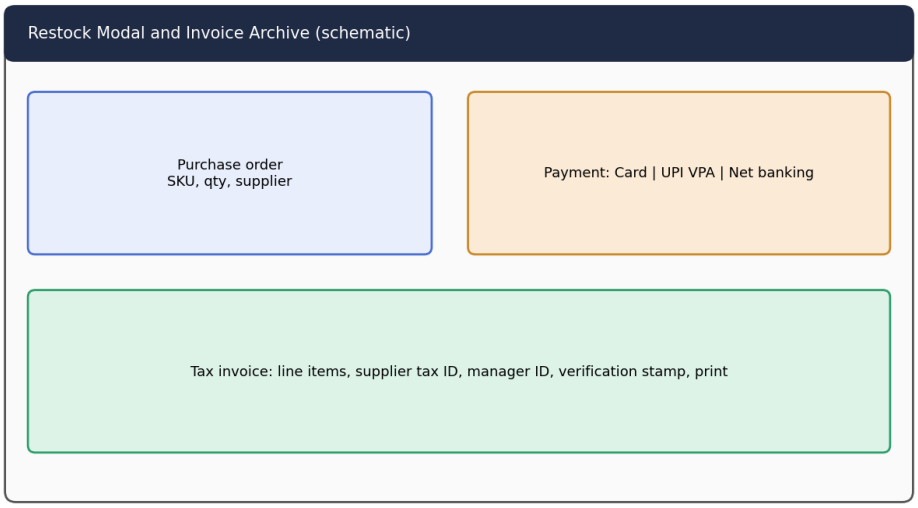


Figure 6.6: Schematic of the Restock Modal and Invoice Archive

6.4 Sample Telemetry Packet

Telemetry travels as compact JSON diff packets. The listing below shows a representative packet for a confirmed stockout. It is an illustration of the structure and not a captured log. Only fields that changed are sent, and each packet carries the camera, the event type and the active operator.

```
{
  "ts": "2026-10-12T10:42:07Z",
  "layer": "edge",
  "camera": "CAM 07",
  "event": "stockout_confirmed",
  "sku": "CER-0412",
  "shelf": { "aisle": 7, "bay": 3, "level": 2 },
  "visual_count": { "prev": 1, "now": 0 },
  "absent_for_s": 95,
  "confidence": 0.91,
  "operator": "harish_manager"
}
```

A POS packet for a checkout has the same envelope with a different event type and the cart contents, and an authentication packet carries the operator and the role. A consistent envelope makes it easy for the drawer, the audit report and the MCP tools to handle every event in the same way.

6.5 Summary

This chapter described the navigation of the cockpit and walked through six scenarios that exercise the closed loop: live surveillance, phantom stockout versus pickup, self-checkout, loss prevention, analytics and restocking with invoicing. The scenarios show how a single action propagates through the shared state and is recorded in the telemetry drawer.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project set out to close the gap between what a retail store's cameras can see, what its POS records say and what its procurement process does. We built a CV-Based Sales Analytics Engine as a four-layer edge-to-cloud pipeline with a vanilla ES6 operations cockpit, using YOLOv11 and COCO-SSD for detection, Firebase and MCP for synchronisation, and an agent engine for temporal filtering, shrinkage analysis and forecasting.

The results show that the approach works as a coherent system. The hysteresis filter separates pickups from stockouts, the shrinkage formula and its behavioural triggers turn depletion and sales into an explainable signal, the self-checkout kiosk feeds the same stock state as the shelf cameras, and the procurement modal turns a safety-stock breach into a purchase order, a simulated payment and a signed tax invoice. Role-based access and operator tagging make each step accountable, and the one-click audit report summarises the day. The architecture keeps inference at the edge and sends only telemetry diffs, which preserves privacy and bandwidth.

7.2 Limitations

- Detection quality depends on camera placement, lighting and occlusion. Reflective packaging remains difficult, and the calibration studio reduces but does not remove this effect.
- Payment channels and tax invoices are simulated, so the system is a demonstration of the workflow and not a certified financial system.
- Operator sessions are stored in LocalStorage, which is suitable for a demonstration but not for a production deployment.
- The throughput, latency and conversion figures are design targets and analytical results from the project specification. They have not been benchmarked on a full multi-camera store deployment.
- The hysteresis dwell duration and the loitering threshold are calibrated settings, and they need tuning for each store and product category.
- A positive shrinkage delta is a risk signal and not proof of theft; every alert needs human confirmation.

7.3 Future Scope

- Benchmark the YOLOv11 TensorRT backend on a Jetson Orin Nano with multiple live RTSP streams, and report measured frame rates, latency and detection accuracy per product class.
- Fine-tune the detector on a store-specific dataset collected through the calibration studio, and measure the improvement on reflective and occluded products.
- Replace the simulated payment and invoice flows with real payment-gateway and tax-invoice integrations, and replace LocalStorage sessions with server-side authentication.
- Learn the hysteresis dwell duration per product category from data instead of setting it by hand.
- Extend forecasting from current sales velocity to models that account for time of day, promotions and seasonality.
- Add an automated test suite covering the detection adaptors, the hysteresis filter, the shrinkage calculation and the checkout flows.
- Evaluate the privacy impact of the shopper-tracking features and add configurable anonymisation and retention policies.

7.4 Summary

This chapter concluded that perception, transaction and procurement can be unified in one low-cost web application, listed the limitations of the current work, and outlined future directions: hardware benchmarking, store-specific fine-tuning, production payment and authentication, learned dwell thresholds and automated testing.

CHAPTER 8
REFERENCES

[1] M. Furmakiewicz et al., “Retail AI copilots,” 2024 (as reviewed in the Review 2 literature survey).

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[3] W. Liu et al., “SSD: Single shot multibox detector,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2016, pp. 21–37.

[4] National Retail Federation, “National Retail Security Survey,” annual report, Washington, DC, USA.

[5] T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.

[6] D. Smilkov et al., “TensorFlow.js: Machine learning for the web and beyond,” in *Proc. Conf. Syst. Mach. Learn. (SysML)*, 2019.

[7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.

[8] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2017, pp. 3645–3649.

[9] G. Jocher, J. Qiu, and A. Chaurasia, “Ultralytics YOLO11,” Ultralytics, 2024. [Online]. Available: https://docs.ultralytics.com

[10] Y. Zhang et al., “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022.

[11] World Wide Web Consortium, “Web Audio API,” W3C Recommendation. [Online]. Available: https://www.w3.org/TR/webaudio/

[12] Google, “Firebase documentation: Cloud Firestore, Cloud Functions and the Firebase MCP server.” [Online]. Available: https://firebase.google.com/docs

[13] Anthropic, “Model Context Protocol specification.” [Online]. Available: https://modelcontextprotocol.io

CHAPTER 9
APPENDIX

9.1 Data Record Formats

Tables 9.1, 9.2 and 9.3 list representative fields stored for the main records. The names describe the content of each record and may differ slightly from the identifiers used in the source files.

Field	Description
sku_id	Unique stock keeping unit identifier
name, category	Product name and taxonomy class
shelf_coordinates	Aisle, bay and level on the planogram
visual_count	Units currently detected by the vision layer
safety_buffer	Minimum level that triggers a reorder
pos_velocity	Daily units sold according to POS
tax_rate	5% for staple foods or 18% for luxury and cosmetics

Table 9.1: Inventory (SKU) Record Fields

Field	Description
camera_id	CAM 02, CAM 04, CAM 07 or CAM 14
aisle	Aisle or concourse covered by the camera
asset	Static test image or live stream source
products	Product classes expected in the view
overlays	Boxes, planogram guidelines, heatmap, trails

Table 9.2: Camera Record Fields

Field	Description
invoice_id	Unique tax invoice number
supplier, supplier_tax_id	Supplier name and tax identification
line_items	SKU, quantity, unit price and tax per line
payment_channel	Card, UPI VPA or net banking
authorised_by	Identifier of the approving manager
verification_stamp	Signed stamp shown on the invoice

Table 9.3: Procurement Invoice Record Fields

9.2 Project Directory Structure

```
CV-Sales-Analytics-Engine/
|-- index.html            main single-page dashboard
|-- test.html             testing mirror of index.html
|-- styles.css            dark cyberpunk / glassmorphic design system
|-- app.js                orchestrator, tab routing, event bus
|-- PROJECT_SUMMARY.md    architecture and functional documentation
|-- assets/
|   |-- camera_cereal.jpg     CAM 07  dry groceries and cereals
|   |-- camera_dairy.jpg      CAM 04  beverages and dairy
|   |-- camera_perfume.jpg    CAM 02  fragrances and cosmetics
|   `-- camera_traffic.jpg    CAM 14  overhead concourse
|-- components/
|   |-- visionFeed.js         detection engine and camera wall
|   |-- selfCheckout.js       self-checkout cart and payment
|   |-- inventoryAndPos.js    stock and POS fusion table
|   |-- lossPrevention.js     shrinkage, loitering, theft
|   |-- analyticsDashboard.js KPIs, canvas charts, runout
|   |-- restockModal.js       purchase order and payment modal
|   |-- invoicesTab.js        procurement and invoice archive
|   |-- invoiceModal.js       certified tax invoice modal
|   |-- userRoleModal.js      login and registration modal
|   |-- trainingModal.js      calibration and fine-tuning studio
|   `-- vivaGuide.js          Review 2 viva defence hub
|-- data/
|   |-- retailData.js         cameras, catalogue, analytics, viva data
|   |-- productKnowledgeBase.js  taxonomy and packaging heuristics
|   `-- authManager.js        credentials, registration, session
`-- utils/
    |-- reportGenerator.js    printable daily audit report
    `-- soundEffects.js       Web Audio synthesiser
```

9.3 Main Commands

Because the application uses native ES6 modules, it must be served over HTTP and not opened from the file system. Any static web server is sufficient.

```
# Serve the project folder (Python 3)
python3 -m http.server 8080

# Open the dashboard in a modern browser
http://localhost:8080/index.html

# Pre-seeded operator profiles
harish_manager
admin
```

Browser webcam detection requires the page to be served from localhost or over HTTPS, and the browser will ask for camera permission when the webcam is first connected.

9.4 Key Parameters and Thresholds

Table 9.4 collects the numerical parameters that appear throughout the report, so that they can be found in one place.

Parameter	Value	Used in
Edge inference target	30 to 45 FPS	Layer 1, vision feed
Synchronisation round-trip target	Under 15 ms	Layer 2, telemetry
Dashboard canvas rendering	60 FPS	Analytics dashboard
Conversion dwell threshold	2.5 minutes (84% conversion)	Analytics, conversion model
Loitering threshold	More than 4.5 minutes	Loss prevention
Tax rate, staple edible foods	5%	Self-checkout
Tax rate, luxury fragrances and cosmetics	18%	Self-checkout
Hysteresis dwell duration T_dwell	Calibrated per category	Inventory and POS fusion
Camera channels	CAM 02, 04, 07, 14	Vision feed
Product classes in taxonomy	10	Training studio

Table 9.4: Key Parameters and Thresholds

CHAPTER 10
SOURCE CODE

10.1 Overview of the Source Code

The complete source of the application is contained in the project directory described in Section 9.2. This chapter gives reference listings for the core algorithms described in Chapter 4: the hysteresis filter, the shrinkage calculation, the tax and bill calculation, the runout forecast and the procedural sound engine. The listings express the logic of the equations in compact JavaScript form so that they can be read alongside the text. They are condensed reference versions, and the full implementations, with user interface wiring, are in the corresponding component files.

10.2 Hysteresis Filter (inventoryAndPos.js)

The filter keeps, for every SKU, the time at which the current run of absence began. A stockout is confirmed only when the run lasts at least the calibrated dwell duration, as in Equation (1).

```
const T_DWELL_MS = 60000;           // calibrated per category in practice
const absentSince = new Map();      // sku -> timestamp of first absent frame

export function updateShelfState(sku, visualCount, now = Date.now()) {
  if (visualCount > 0) {            // item visible: reset, treat as pickup
    absentSince.delete(sku);
    return "IN_STOCK";
  }
  if (!absentSince.has(sku)) absentSince.set(sku, now);
  const absentFor = now - absentSince.get(sku);
  return absentFor >= T_DWELL_MS ? "CONFIRMED_STOCKOUT" : "CANDIDATE_GAP";
}
```

10.3 Shrinkage Calculation (lossPrevention.js)

The function implements Equation (2) and combines the result with the behavioural triggers before raising an incident.

```
export function shrinkageDelta({ visualDepletion, posCount, writeOffs }) {
  return visualDepletion - (posCount + writeOffs);
}

export function classifyIncident(delta, signals) {
  const { unscannedExit, wrongCategory, stationaryMinutes } = signals;
  if (delta > 0 && unscannedExit) return "UNSCANNED_CONCEALMENT";
  if (wrongCategory)              return "PLANOGRAM_BREACH";
  if (stationaryMinutes > 4.5)    return "LOITERING";
  return delta > 0 ? "VARIANCE_REVIEW" : "NONE";
}
```

10.4 Tax, Bill and Runout (selfCheckout.js, analyticsDashboard.js)

These functions implement Equations (3) and (4). The tax rate is looked up from the product class.

```
const TAX = { staple: 0.05, luxury: 0.18 };

export function lineTotal(price, qty, taxClass) {
  return price * qty * (1 + TAX[taxClass]);
}

export function billTotal(cart) {
  return cart.reduce((s, l) => s + lineTotal(l.price, l.qty, l.taxClass), 0);
}

export function runwayHours(currentStock, unitsPerHour) {
  return unitsPerHour > 0 ? currentStock / unitsPerHour : Infinity;
}
```

10.5 Procedural Sound Engine (soundEffects.js)

Sounds are generated from oscillators and gain envelopes with the Web Audio API [11], so no audio files are needed.

```
const ctx = new (window.AudioContext || window.webkitAudioContext)();

export function beep(freq = 880, ms = 120, type = "sine", vol = 0.15) {
  const osc = ctx.createOscillator();
  const gain = ctx.createGain();
  osc.type = type;
  osc.frequency.value = freq;
  gain.gain.setValueAtTime(vol, ctx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.0001, ctx.currentTime + ms / 1000);
  osc.connect(gain).connect(ctx.destination);
  osc.start();
  osc.stop(ctx.currentTime + ms / 1000);
}

export const alarm = () => { beep(1200, 250, "square"); beep(800, 250, "square"); };
```

10.6 Event Bus (app.js)

The event bus is a small publish and subscribe object through which the components communicate. Every published event is also tagged with the active operator and mirrored to the telemetry drawer.

```
const handlers = {};
export const bus = {
  on(evt, fn) { (handlers[evt] ||= []).push(fn); },
  emit(evt, payload) {
    const packet = { ts: new Date().toISOString(), event: evt,
                     operator: currentOperator(), ...payload };
    drawerLog(packet);
    (handlers[evt] || []).forEach(fn => fn(packet));
  },
};
```

CHAPTER 11
VIVA DEFENCE QUESTIONS AND ANSWERS

11.1 Overview

The viva defence hub of the application (components/vivaGuide.js) contains question cards that prepare the presenter for the Review 2 examination. This chapter reproduces the main questions and the technical justifications in the form of short answers. They follow the same themes as the hub: model selection, occlusion mitigation, temporal filtering and protocol security.

11.2 Model Selection

Question: Why were YOLOv11 and COCO-SSD both used? Answer: The two backends serve different roles. YOLOv11, exported to TensorRT, is intended for continuous monitoring on an edge device because it targets 30 to 45 frames per second on embedded hardware. COCO-SSD running in TensorFlow.js needs no installation and keeps the image on the client, which makes it the right choice for demonstration and calibration. Both produce the same bounding-box and confidence structure, so the cockpit does not depend on which one is used.

11.3 Occlusion Mitigation

Question: How does the system cope with occluded or partly hidden products? Answer: Occlusion is handled at three levels. At the detection level, the product knowledge base records packaging cues such as aspect ratio and material, so a partly visible product can still be classified. At the time level, the hysteresis filter avoids declaring a stockout because of a momentary occlusion by a shopper. At the sensing level, barcode and QR ingest provide an independent optical confirmation, and the overhead camera gives a second viewpoint on the same aisle.

11.4 Hysteresis Filter

Question: Why is a hysteresis filter necessary? Answer: A detector reports what is visible in a frame, and a shopper who lifts a product to read its label creates a gap that looks like a stockout. Without a temporal rule, every such event would raise a replenishment alert and staff would stop trusting the system. The filter requires the product to remain absent for a calibrated dwell duration, which removes transient pickups while still catching real gaps. The cost is a short delay in confirming a genuine stockout, which is why the dwell duration is tuned per category.

11.5 Shrinkage Formula

Question: How is shrinkage quantified, and does a positive value prove theft? Answer: Shrinkage delta is the visual shelf depletion minus the sum of POS checkout count and documented write-offs. A positive value means that more items left the shelf than are explained by sales and write-offs. It does not prove theft, because detection error and relocated items can contribute. The system therefore combines the number with behavioural triggers and presents the result as an incident for a person to review.

11.6 Edge-to-Cloud Design and MCP Safeguards

Question: Why keep inference at the edge, and how does the MCP server protect the system? Answer: Inference at the edge avoids streaming uncompressed video to the cloud, which reduces bandwidth and protects privacy, and the system sends only JSON diff packets. The MCP server exposes the retail state through typed, schema-described tools, so an agent can perform only declared actions and every call is validated. This is important because some tools, such as creating a purchase order, have financial effects.

11.7 Research Gap and Contribution

Question: What is the novelty of the work? Answer: Prior work solves isolated sub-problems: detection accuracy, footfall analytics, loss prevention or copilots. The contribution here is to close the loop between raw video pixels, POS transactions and automated procurement in one application, with a temporal filter that keeps alerts trustworthy and an access-control layer that makes every action attributable.

11.8 Summary

This chapter summarised the main viva questions on model selection, occlusion, hysteresis, shrinkage, edge-to-cloud design and novelty, with concise technical justifications that match the design described in the earlier chapters.